PROJECT SHEET 4 · TIED TO YOUR REPO'S CODE

UVM Core + Project Deep-Dive Q&A

The UVM mechanics your repo uses, explained at the level a senior interviewer pushes to — then the questions they'll ask about your code.

1 Phases

build	connect	end_of_elab	start_of_sim	run	extract	check	report	final
top-down	bottom-up	print topology	banners	task · all in parallel		SB final checks	summary	

Why build is top-down: a parent must exist and set config before children are created. **Connect is bottom-up:** children's ports must exist first.

Only run (and its 12 sub-phases: reset, configure, main, shutdown + pre/post) consumes time. All others are functions.

Your repo: SB checks in `check_phase`, coverage prints in `report_phase`, base test prints topology in `end_of_elaboration_phase` — all correct placements.

2 Factory

```
`uvm_component_utils(axi4_driver)   // register
drv = axi4_driver::type_id::create("drv", this);

// in a test — swap without editing env code
axi4_driver::type_id::set_type_override(
    axi4_err_driver::get_type());
// or only one instance
axi4_driver::type_id::set_inst_override(
    axi4_err_driver::get_type(),
    "env_h.agent_h.driver_h", this);
```

- Calling `new()` directly bypasses overrides — the #1 factory mistake.
- Override must be set **before** the target is created (test's build_phase, before `super` builds env).
- Debug: `factory.print()`.

3 config_db

```
// tb_top (missing in your repo!)
uvm_config_db#(virtual axi4_if)::set(
    null, "uvm_test_top.*", "AXI4_VIF", bus);
run_test();

// driver / monitor build_phase
if (!uvm_config_db#(virtual axi4_if)::get(
        this, "", "AXI4_VIF", vif))
  `uvm_fatal("NOVIF", "vif not set")
```

- Key = context + instance path + field name + **type**. A type mismatch silently fails the get.
- During build, a set from **higher in the hierarchy wins**; at the same level, last set wins.
- Debug: `+UVM_CONFIG_DB_TRACE`.

4 Objections & end of test

```
task run_phase(uvm_phase phase);
  phase.raise_objection(this);
  seq.start(env_h.agent_h.sequencer_h);
  phase.drop_objection(this);
endtask

// instead of the repo's fixed #100ns:
function void end_of_elaboration_phase(uvm_phase p);
  uvm_phase run = uvm_run_phase::get();
  run.get_objection().set_drain_time(this, 200ns);
endfunction
```

- run ends when every objection is dropped (plus drain time).
- Drain time lets the last responses reach the scoreboard.
- Better still: SB raises an objection while it has pending expected items.

5 TLM — the paths in your env

Piece	Semantics	In your repo
analysis port	1 → many, `write()` is a non-blocking **function**	`monitor.ap_write / ap_read`
uvm_tlm_analysis_fifo	Unbounded buffer; subscriber does blocking `get()` in its own thread	Scoreboard `wr_fifo / rd_fifo`
analysis imp	Calls the owner's `write()` immediately	Coverage `wr_export / rd_export`
`` `uvm_analysis_imp_decl(_x) ``	Creates `uvm_analysis_imp_x` → method `write_x()`; needed for >1 imp in one class	**Missing** — add `(_write)` and `(_read)`
seq_item_port / export	Pull interface driver ↔ sequencer	agent connect_phase

FIFO vs imp: the scoreboard needs to *wait* and match (thread + blocking get → FIFO). Coverage just samples on arrival (function → imp).

6 Sequence ↔ driver handshake

```
// sequence body()                  // driver run_phase
start_item(txn);       // wait grant   seq_item_port.get_next_item(txn);
txn.randomize() with {...};          drive(txn);
finish_item(txn);      // wait done   seq_item_port.item_done();
// optional response path
get_response(rsp);                     rsp.set_id_info(txn);
                                       seq_item_port.put_response(rsp);
```

- **Randomize between start_item and finish_item** — late randomization can react to sequencer state.
- `get_next_item` + `item_done` vs `get` (implicit item_done) vs `try_next_item` (non-blocking, for idle cycles).
- Multiple sequences on one sequencer → **arbitration** (FIFO, weighted, random, strict…); `lock` / `grab` for exclusive access.
- **Virtual sequence** on a virtual sequencer coordinates several agents; `` `uvm_declare_p_sequencer `` gives typed access.
- Your repo's driver writes `bresp / rdata` back into the same item — works because the sequence holds that handle after finish_item.

7 object vs component

	uvm_object	uvm_component
Examples	seq_item, sequence, config	driver, monitor, env
Lifetime	Created & dropped any time	Built once, lives all sim
Phases / hierarchy	No	Yes (parent, path)
Macro	uvm_object_utils	uvm_component_utils

Your item uses hand-written `do_copy / do_compare / do_print` instead of field macros — faster and easier to debug; say that's deliberate.

8 Interface, clocking block, modport

```
clocking driver_cb @(posedge aclk);
  default input #1step output #1;
  output awaddr, awvalid, wdata, wstrb,
         wvalid, bready, araddr, arvalid, rready;
  input  awready, wready, bvalid, bresp,
         arready, rvalid, rdata, rresp;
endclocking
clocking monitor_cb @(posedge aclk);
  default input #1step;
  input awaddr, awvalid, awready, /* … all */;
endclocking
modport DRV (clocking driver_cb, input aresetn);
modport MON (clocking monitor_cb, input aresetn);
```

Your driver/monitor reference these, but the interface doesn't define them yet — this is the block to add.

9 Other must-knows

- **Callbacks:** hook into a driver (inject errors) without subclassing.
- **Report catcher:** demote an expected UVM_ERROR in a negative test (e.g. SLVERR test).
- **Verbosity per component:** `+uvm_set_verbosity=…` to debug one block.
- **Transaction recording** to see items on the waveform.
- **uvm_event / uvm_barrier** for cross-component sync.
- **Agent active/passive:** `is_active` via config_db; passive agents reuse your monitor at SoC level.

? Questions an interviewer will ask about *your* repo

Question	Strong answer
Why does the monitor `create()` a new item every loop?	Analysis ports pass handles; reusing one object would overwrite items the SB hasn't processed yet.
Why separate write and read analysis ports?	Independent checker threads per direction; could merge with txn_type, but separate keeps each checker simple.
Why tlm_analysis_fifo in the SB but imps in coverage?	SB waits/blocks per thread; coverage just samples in a function.
When would you make the agent passive?	When another master drives the bus (SoC level) — reuse the monitor, SB and coverage.
Why does `do_compare` skip rdata/resp?	Those are outputs; correctness of outputs is the scoreboard's job against the model.
Why use a clocking block?	Removes TB/DUT races: inputs sampled just before the edge, outputs driven just after.
The SB skips never-written words. Risk?	Reads after reset go unchecked; the RAM resets to 0, so model that and check.

Question	Strong answer
What did your coverage miss?	Handshake order and stalls — exactly where the W-before-AW deadlock hid. Then show the new protocol covergroup.
How would you verify 4 outstanding transactions?	AXI-Lite slave here allows 1; for AXI4 use per-ID queues in the SB (sheet 7) and an outstanding-depth coverpoint.
A test fails on seed 1234 only. Steps?	Rerun that seed, raise verbosity on the agent, dump waves around the first error, check whether DUT or model is wrong, add a directed test.
How do you make sure the checker works?	Inject a known bug (e.g. ignore WSTRB) and confirm the SB fails — mutation testing.
How would you add RAL?	Reg block + adapter (reg2bus/bus2reg) + predictor on the monitor; then built-in reset/bit-bash/mem-walk sequences.
Why iverilog — and its limits?	Free and fast to share; no full UVM, covergroups or constraints, so the runnable TB mirrors UVM with tasks. The UVM layer targets commercial simulators.
What would you do with one more week?	SVA bind + formal proof of the handshake properties, then extend to full AXI4 with bursts and IDs.